

# 导表使用说明

- 1 本地导表使用说明:
- 2 导表预处理流程:
  - 2.1 每行数据预处理:
  - 2.2 整张表进行预处理:
  - 2.3 导表依赖检查
  - 2.4 多张数据表进行预处理
  - 2.5 多语言预处理
- 3 C#代码中使用
- 4 后续待完善:

## 本地导表使用说明:



### 提示

- 安装Python3以上版本
- 导表工具目录在: [https://svn.tap4fun.com/svn/x3/design/Tools/table\\_exporter](https://svn.tap4fun.com/svn/x3/design/Tools/table_exporter), 如果已经申请design的svn, 直接更新就可以了
- 进入table\_exporter目录
- 安装必须的Python第三方库: `pip install -r .\requirements.txt`
- 执行导表的bat脚本:
  - 如果不需要C#调试, 可以直接执行`export_table.bat`, 没有报错, 就可以将相关修改提交到SVN
  - 如果需要C#调试: 执行`export_table_and_cp_to_client.bat`, 会将相关的二进制文件、C#导出都会拷贝到client对应的目录, 调试完成以后, 可以将相关修改提交到SVN。在Client目录下生成的中间文件**不要上传**

## 导表预处理流程:



### 提示

- 需要在def/文件夹下, 增加Excel中以`sheetName_def.py`文件。注意的是`sheetName`必须全部小写
- 导表预处理支持: **每行数据预处理**和**整张表进行预处理**

下面分别介绍两种预处理:

## 每行数据预处理:



### 提示

- 需要在def文件中使用, **@converter**装饰器装饰处理函数, 处理函数必须要**返回item**:

```
@converter
def user_convert_func(item):
    return item
```



### 提示

- `item`中包含这行数据的所有属性值, 可以直接使用该值, 属性名对应Excel中的第六列中的属性名。详见下图: `item.ID`、`item.Worker1`等等。
- 属性名: **属性名包含\_字符的会自动去掉\_字符**。 eg: `DK_Icon`会变成`DKIcon`, 使用的时候注意一下。
- 增加属性: 必须调用`setAttrNameFieldDef`方法, 设置好相关属性信息后, 就可以直接设置该属性数值了。`setAttrNameFieldDef`需要传入4个参数:
  - 第一个参数: 属性名。该属性名首字母必须大些。
  - 第二个参数: Domain。 标明这个字段是服务器、客户端、还是服务器/客户端都需要。
  - 第三个参数: 数据转换的格式。目前支持的数据转换格式, 可以参看`FieldTypes.py`中的定义。`Int`、`Long`、`Time`、`Dict`、`List`、`Float`、`Str`、`Bool`、`ListInnerMessage`、`DictInnerMessage`、还有`CustomMessageType`。如果需要`List`嵌套`List`、`Dict`嵌套`Dict`等需要自定义`CustomMessageType`, 但是为了简化这些操作, 封装了`ListInnerMessage`、`DictInnerMessage`。`CustomMessageType`一般是用不到, 如果需要的话, 可能数据格式太复杂了, 可以先与**保成**沟通一下。
  - 第四个参数: 注释。会在生成C#代码的时候, 显示该注释信息。

```
item.setAttrNameFieldDef("WorkerLevels", DOMAIN_ALL, Dict(Int(), Int()), "")
item.WorkerLevels = WorkerLevels
```

#### 提示

- 删除属性: 直接调用`delattr`, 这一条数据就会被自动删除, proto中也不会被导出。
- CustomMessae定义: 需要传入MessageName, `[[DOMAIN, FieldName, Convert, Comment], [DOMAIN, FieldName2, Convert, Comment], ...]`, 自定义Message格式是一个Dict, 按照Field格式进行赋值就可以了

```
item.setAttrNameFieldDef("Test", DOMAIN_ALL, CustomMessageType("CTestMessage", [[DOMAIN_ALL, "HH", List(Int()), "test"]]), "")
```

```
item.Test = {}
item.Test["HH"] = [1, 2, 3]
```

#### 提示

- ListInnerMessage是为了解决简单的List嵌套List (嵌套List<int、long、string、bool、float>)等封装出来的。如果嵌套的List是复杂的数据结构, 还是需要使用CustomMessage来定义。ListInnerMessage使用和List类似, 在数据处理的时候, **必须设置对应的Vals**。

```
...
groupID = item.Group
if groupID not in GroupStorageTypes:
    GroupStorageTypes[groupID] = {}
    GroupStorageTypes[groupID]["Vals"] = StorageType # TODOVals

row = otable.addRow("GroupStorageTypes", DOMAIN_ALL, Dict(Int(), ListInnerMessage(Int())), "")
setattr(row, "GroupStorageTypes", GroupStorageTypes)
```

#### 提示

- DictInnerMessage是为了解决简单的Dict嵌套Dict (嵌套Dict<keyType: string、int、long, ValueType: int、long、string、bool、float>)等封装出来的。如果嵌套的Dict是复杂的数据结构, 还是需要使用CustomMessage来定义。DictInnerMessage使用和Dict类似, 在数据处理的时候, **必须设置对应的Vals**。
- 每行处理数据的例子, 可以参考blueprints\_def.py, 该功能是将蓝图表中的, 一些属性数据, 合并转换成Dict类型, 并将表中不需要的数据删除:

```
@converter
def user_convert_func(item):
    WorkerLevels = {}
    for i in range(WORKER_NUM):
        index = i + 1
        workerID = getattr(item, WORKER_FORMAT % index)
        level = getattr(item, WORKER_LEVEL_FORMAT % index)
        if workerID > 0:
            WorkerLevels[workerID] = level
        delattr(item, WORKER_FORMAT % index)
        delattr(item, WORKER_LEVEL_FORMAT % index)

    item.setAttrNameFieldDef("WorkerLevels", DOMAIN_ALL, Dict(Int(), Int()), "")
    item.WorkerLevels = WorkerLevels
    return item
```

| J       | K            | L       | M            | N       | O            |
|---------|--------------|---------|--------------|---------|--------------|
| cs      | cs           | cs      | cs           | cs      | cs           |
| int     | int          | int     | int          | int     | int          |
|         |              |         |              |         |              |
|         |              |         |              |         |              |
|         |              |         |              |         |              |
| 需求工人1   | 需求工人1等级      | 需求工人2   | 需求工人2等级      | 需求工人3   | 需求工人3等级      |
| Worker1 | Worker1Level | Worker2 | Worker2Level | Worker3 | Worker3Level |
| 130111  | 1            |         | 1            |         | 1            |
| 130111  | 2            |         | 1            |         | 1            |
| 130111  | 2            |         | 1            |         | 1            |
| 130111  | 5            |         | 1            |         | 1            |
| 130111  | 9            |         | 1            |         | 1            |
| 130111  | 13           | 130119  | 1            |         | 1            |
| 130111  | 17           | 130119  | 13           |         | 7            |
| 130111  | 21           | 130119  | 17           |         | 11           |
| 130111  | 23           | 130119  | 19           |         | 16           |
| 130111  | 25           | 130119  | 21           | 130115  | 18           |
| 130111  | 27           | 130119  | 23           | 130115  | 20           |
| 130111  | 29           | 130119  | 25           | 130115  | 22           |

属性名

整张表进行预处理:



提示

- 需要在def文件中使用，**@convertTable**装饰器装饰处理函数，处理函数必须要返回otable:

```
@convertTable
def table_convert_func(otable):
    return otable
```



提示

- otable.table存储每一行数据，可以遍历每行数据，然后进行整张表的数据预处理
- 预处理完，可以将相关插入新行数据到otable，需要调用otable.addNewRow，该函数也需要4个参数，参数和setattrNameFieldDef是一样的，在此就不详细介绍了
- 整行预处理的数据，属性名不能是Configs，这个字段预留处理原来每行数据的(使用该字段会报错哈)
- 整张表进行预处理例子，可以参考item\_def.py。下面的例子是遍历物品表，统计货币ItemIDs和资源ItemIDs，然后设置新行数据:

```

@convertTable
def table_convert_func(otable):
    MoneyItemIDs = []
    ResItemIDs = []
    for item in otable.table:
        if item.Type == MONEY_TYPE:
            MoneyItemIDs.append(item.ID)
        elif item.Type == RES_TYPE:
            ResItemIDs.append(item.ID)

    #
    row = otable.addNewRow("MoneyItemIDs", DOMAIN_ALL, List(Int()), "")
    row.MoneyItemIDs = MoneyItemIDs

    row = otable.addNewRow("ResItemIDs", DOMAIN_ALL, List(Int()), "")
    setattr(row, "ResItemIDs", ResItemIDs)

    return otable

```

## 导表依赖检查



### 提示

- 一般策划在第三行标记即可。但是有一些数据(eg:家具Type有可能涉及到资源、物品Type等等)，可能牵扯到不同表，如果需要提前检查，可以按照下面方法进行依赖检查。
- 在对应的def文件中，直接调用register\_depend\_config\_and\_val即可。需要传入的两个参数：
  - sheet\_name: 依赖配置表的名称
  - val: 检查的值，只支持int、string、long、List<int>、List<long>、List<string>
- eg: register\_depend\_config\_and\_val("Item", 1001001)，物品表中必须存在1001001这个ID数据，不存在就会报错。

## 多张数据表进行预处理



### 提示

- 多张表的后处理脚本放在PostProcessData中
- 封装两个接口：> - Utils.getTableData: 获取另外一张表的整张表的数据 > - Utils.getTableRowDataByPrimarykey: 获取一行的数据
- 例子：物品进入货架时，依赖蓝图中的Type，不想再添加物品的时候，还需要查蓝图表，因此将Type写入到物品中，以空间换时间：

```

def deal_item_and_blueprints_data():
    itemTableData = Utils.getTableData("Item")
    for item in itemTableData.table:
        if item.getPrimarykey() != "ID":
            continue

        BlueprintID = item.Blueprint
        item.setAttrNameFieldDef("ItemProductType", DOMAIN_ALL, Int(), "")
        item.ItemProductType = 0
        if BlueprintID != 0:
            blueprintItemData = Utils.getTableRowDataByPrimarykey("Blueprints", BlueprintID)
            item.ItemProductType = blueprintItemData.Type

```

## 多语言预处理

- 如果对多语言内容(Name、描述等)进行预处理，需要将预处理代码放到table\_convert\_func阶段，不要在item\_convert\_func中进行预处理，否则会导致一直显示中文的问题。

## C#代码中使用



#### 提示

- 获取每一行数据和之前的接口保持一致，下面以CBBlueprints举例，TODO: !!!特别需要注意的是，**cfg的数据类型和之前的不同，之前的是CBBlueprints。现在返回的类型是CBBlueprintsCfg类型数据**，CBBlueprintsCfg包含每一行数据。

```
var cfg = CBlueprints.I(1001)
```



#### 提示

- 不再提供RawList和RawDict等接口，如果需要遍历整张表的话，看看是否可以在导表中预处理。如果确定需要，可以通过**CBBlueprints.Instance.Configs**获取整张表的信息，Configs里对应的是，表中的每行数据，不包含导表中预处理的数据。
- 获取整张表的预处理数据，可以使用**CBBlueprints.Instance.XX**就可以访问预处理数据。
- Const中的数据，访问需要通过**CConstCfg.Instance.XX**进行访问。

后续待完善：



#### 提示

- 导表数据，需要设置为只读属性，防止在外面修改导表中的数据。
- 多张表数据，导入到一个Table中
- 多语言功能支持
- 客户端、服务器数据分离
- 客户端，每行数据，按需序列化。
- 功能刚开发完成，如果有什么异常的问题，请联系陈保成。



#### 修改历史

0bf55ed9: (2023-08-11 17:37) (cuixiaolin) Merge branch 'dev' of git.tap4fun.com:x3/x3-project into dev

渲染自x3/x3-project仓库的导表使用说明.md文件